

CS-250: Assignment 2

LeetCode Biweekly Contest 193

Muhammad Mujtaba

CMD ID: 540040

mmujtaba.bese25seecs@seecs.edu.pk

Contest URL: https://leetcode.com/contest/biweekly-contest-193

LeetCode account: muhammad_mujtaba_nust

LeetCode email: mmujtaba.bese25seecs@seecs.edu.pk

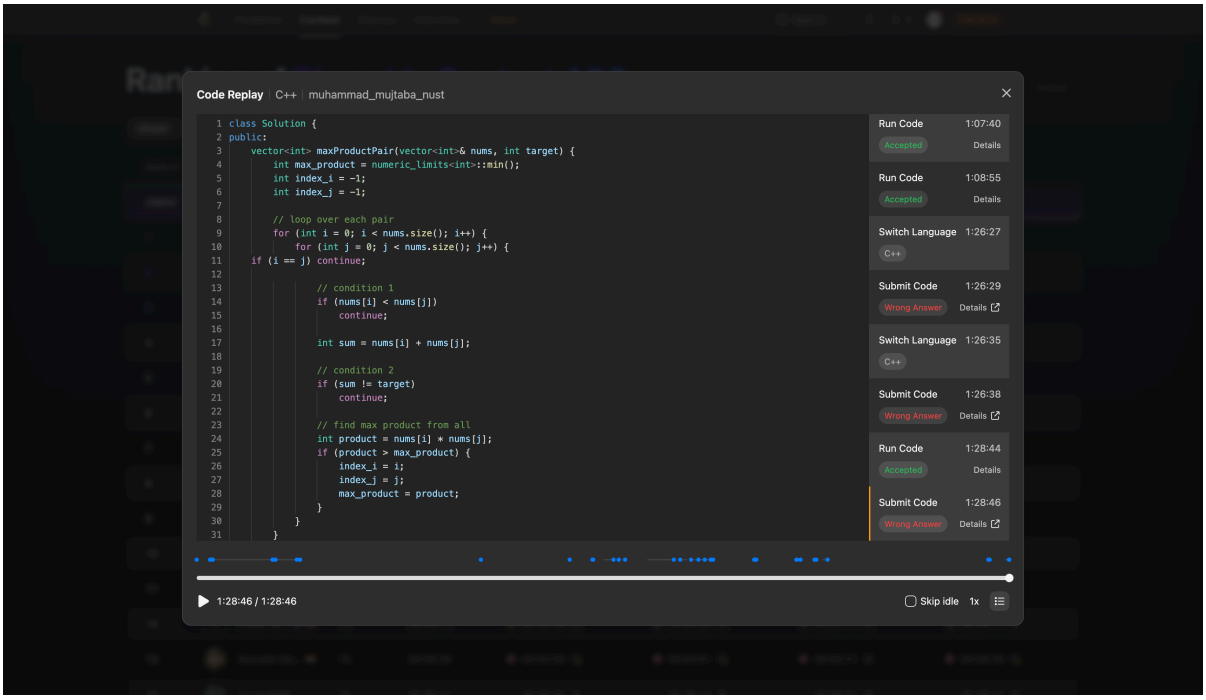
Profile Link: https://leetcode.com/u/muhammad_mujtaba_nust

Class: BESE 16B

Batch: 2k25

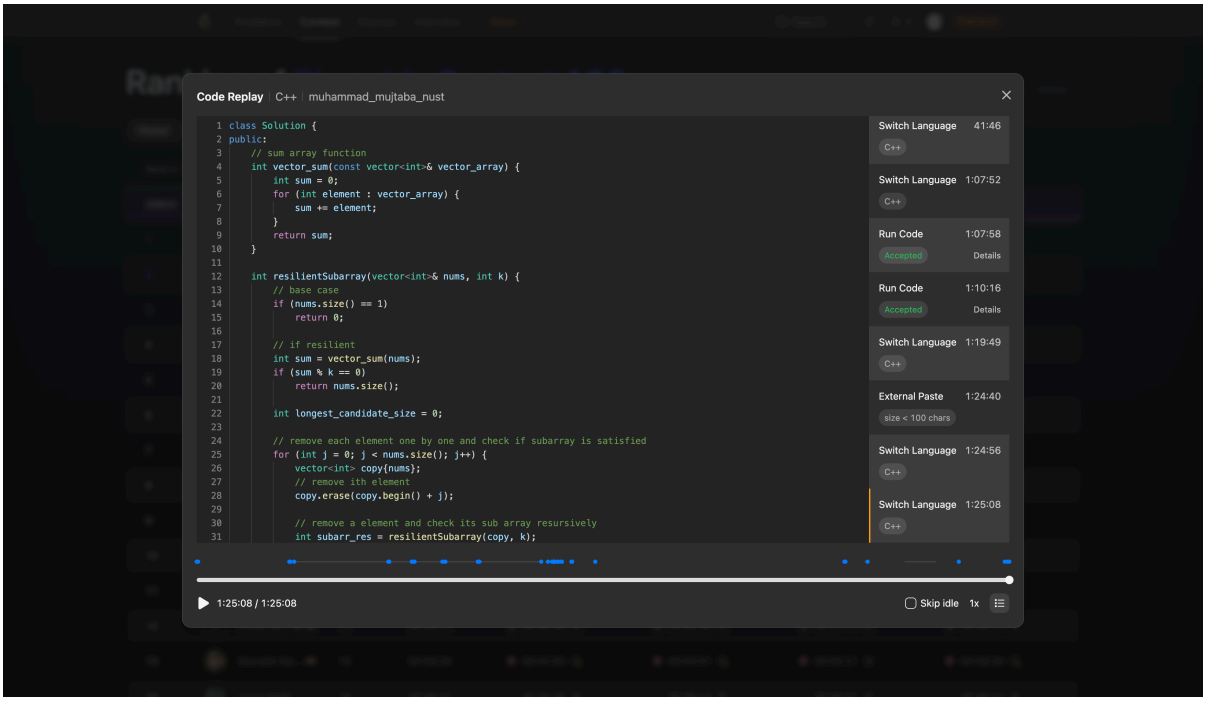
Part 1:

Problem 1:



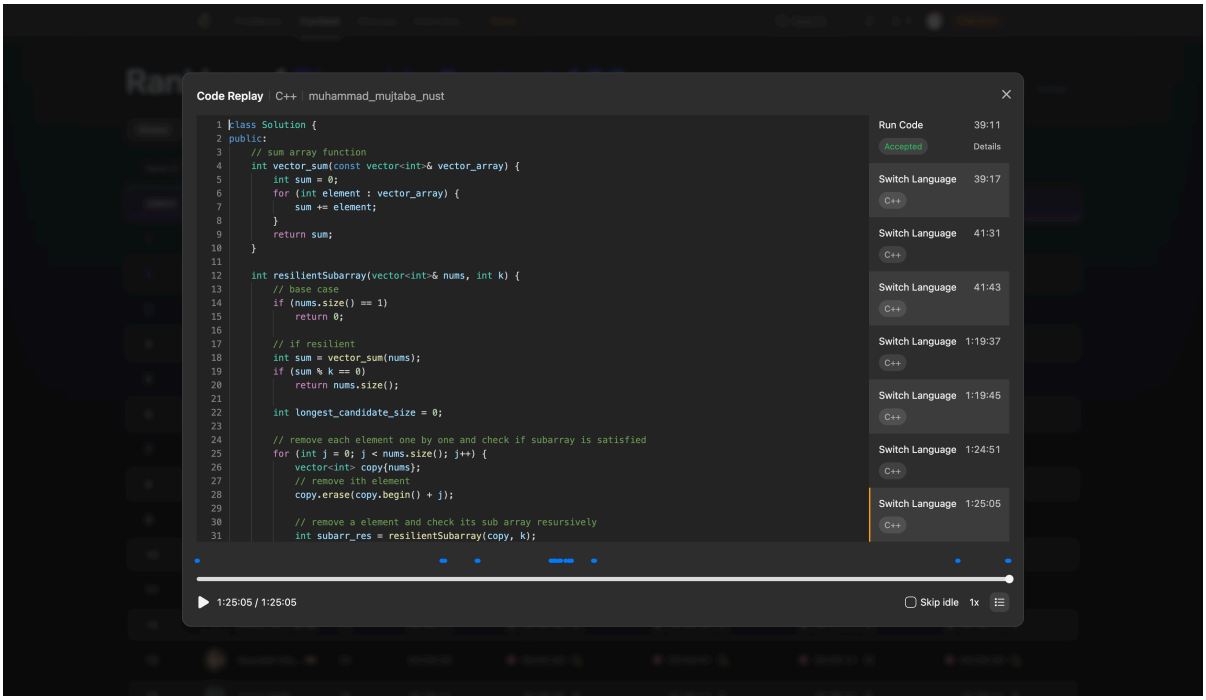
- **Initial Understanding:**
  - Find the largest number pair that sums up to given target having the largest product from each possible pair and return it indices, indices being distinct.
  - First number in pair should be greater than second number.
- **My approach:**
  - Loop over each possible pair
  - If (indices same) or (first value less than second value) or (sum is not target), skip
  - Find product, compare with stored largest, and reassign if new largest found.
- **Data structures used:** Resizable array (vector) provided.
- **Challenges Faced:** Not being able to debug properly
- **Time Complexity:** $O(n^2)$
- **If I had more time:** Make an ordered map <pair → (sum, product)>, then calculate the sum, product in one pass, and return the pair with largest product in one query, get answer in probably O(N) time, O(N) since one pass required to populate the map.

Problem 2:



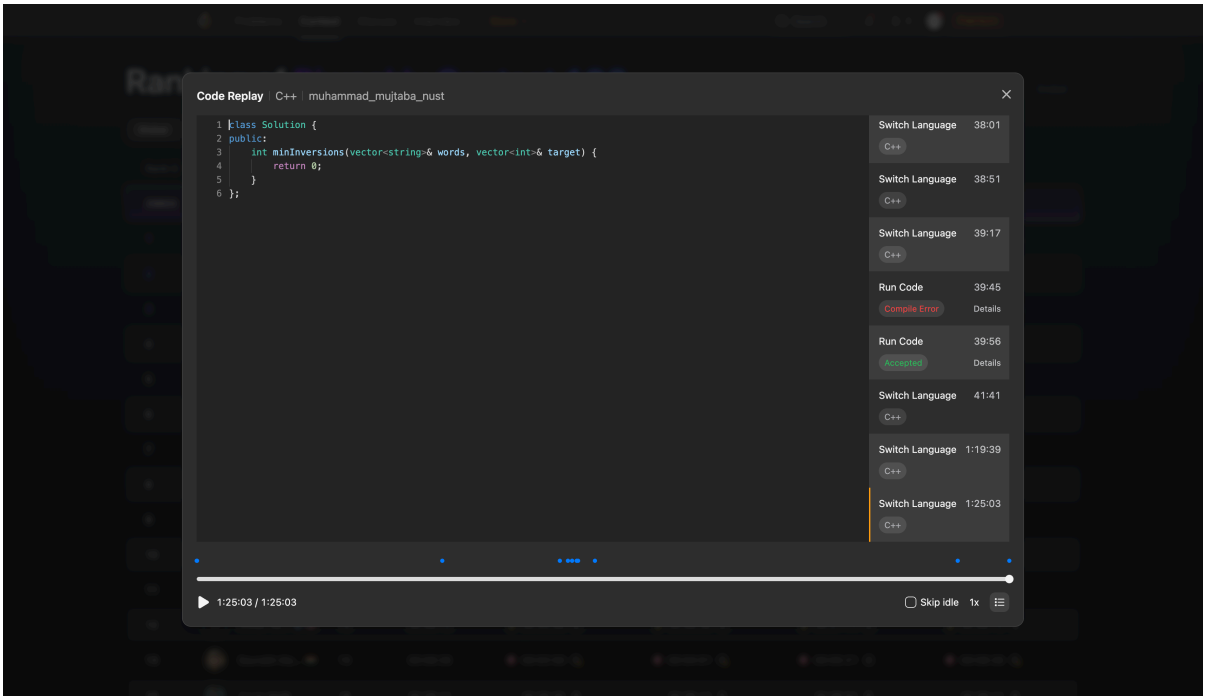
- **Initial Understanding:**
  - Return the length of the longest sub array such that removing a singular item from that sub array, the sum of that sub array is divisible by given k.
  - Such sub array is called a “resilient” sub array.
- **My approach:**
  - If array size is 1, return 0
  - If sum of array is divisible by k, return size of current array.
  - Call recursively for all sub arrays
  - **NOTE:** This solution is not a working solution
- **DSA usage:** Resizable array (vector) with Recursive Programming.
- **Challenges Faced:** Not being able to debug properly.
- **Time Complexity:** O(n!) called once for each possible sub array
- **If I had more time:** Go through layer by layer for the sub arrays, if a sub array is “resilient” then no need to check lower layer. Debug each step, devise a non-recursive solution.

Problem 3:



This problem description was the same as the one above.

Problem 4:



Not enough time to attempt.

Part 2:

Link: https://chatgpt.com/share/6acb912b-fa1c-83ee-ac76-fd26530574b3

Problem 1:

```cpp
class Solution {
public:
    vector<int> maxProductPair(vector<int>& nums, int target) {
        int seen[201];
        fill(seen, seen + 201, -1);

        int best = INT_MIN;
        vector<int> ans = {-1, -1};

        for (int i = 0; i < nums.size(); i++) {
            int x = nums[i];
            int y = target - x;

            if (x != y && y >= -100 && y <= 100 && seen[y + 100] != -1) {
                int j = seen[y + 100];
                int product = x * y;

                if (product > best) {
                    best = product;
                    ans = (x > y) ? vector<int>{i, j} : vector<int>{j, i};
                }
            }

            seen[x + 100] = i;
        }

        return ans;
    }
};
```

- **Approach:**
  - Create an array to store “seen” values
  - Loop over each value in the array, for that value, calculate complement
  - If its complement was already “seen” && the values are different && pair valid
  - If pair valid, update the best product and indices
  - Store the “seen” value
- **Code quality:** Comments could be added, for example why seen array has “+ 100” when accessing or storing (the code takes advantage of given constraints that nums[i] >= -100 to store all elements in a singular array)
- **Optimizations:** Go through in one pass instead of two passes, and keep track of values already visited and use them to get optimal pair
- **Complexity:** O(n)
- **My Strengths:** My solution was more generalized as that has no constraints regarding what inputs were provided.
- **Weakness Identified:**
  - Not reversing the indices if nums[i] > nums[j]
  - Having to visit each possible pair

Problem 2:

```cpp
class Solution {
public:
    int resilientSubarray(vector<int>& nums, int k) {
        int longest = 1;
        int length = 0;
        int previous = -1;

        for (int x : nums) {
            int r = x % k;

            if (r == previous) {
                length++;
            } else {
                length = 1;
            }

            previous = r;

            if ((length - 1) * r % k == 0) {
                longest = max(longest, length);
            }
        }

        return longest;
    }
};
```

- **Approach:**
    - Calculate the remainder of each element when divided by k (in resilient sub array all elements divided by k has the same reminder)
    - Keep track of how many consecutive elements has the same reminder
    - If reminder same then increase “length”, the current longest detected
    - If not reset the reminder
    - Check if array is resilient, (length -1) * reminder is divisible by k
- **Code quality:** Good code quality, but comments would be good. It probably skipped them due to standard LeetCode conventions.
- **Optimizations:** Reduce the problem using mathematics to identify what is resiliency and lower abstracted relation can be used, i.e. for resiliency all elements must have same reminder and what condition is satisfied by array if it is resilient
- **Complexity:** O(n)
- **My Strengths:** None
- **Weakness Identified:**
    - Try to not tackle the problem head on, abstract and search for a better solution, leverage critical thinking and problem solving.

Problem 3:

Same problem statement as above, with input size being greater, leading the product (length – 1) overflowing 32-bit signed integer, 1LL was used to make it “long long” to over overflow.

Problem 4:

```
class Solution {
    struct Node {
        int children[10];
        int wordIndex = -1;

        Node() {
            fill(children, children + 10, -1);
        }
    };

    vector<Node> trie;
    vector<int> pos;
    long long total = 0;

    vector<int> solve(int node) {
        vector<vector<int>> groups;

        // Solve each child subtree.
        for (int child : trie[node].children) {
            if (child != -1) {
                groups.push_back(solve(child));
            }
        }

        int d = groups.size();
        long long cost[10][10] = {};

        // Count inversions between each pair of child subtrees.
        for (int a = 0; a < d; a++) {
            for (int b = a + 1; b < d; b++) {
                int j = 0;

                for (int rank : groups[a]) {
                    while (j < groups[b].size() &&
                           groups[b][j] < rank) {
                        j++;
                    }

                    cost[a][b] += j;
                }

                cost[b][a] =
                    1LL * groups[a].size() * groups[b].size()
                    - cost[a][b];
            }
        }

        // Find the optimal ordering of children.
        int states = 1 << d;
        vector<long long> dp(states, LLONG_MAX);
        dp[0] = 0;

        for (int mask = 1; mask < states; mask++) {
            for (int last = 0; last < d; last++) {
                if (!(mask & (1 << last))) {
                    continue;
                }

                int rest = mask ^ (1 << last);
                long long extra = 0;

                for (int before = 0; before < d; before++) {
                    if (rest & (1 << before)) {
                        extra += cost[before][last];
                    }
                }

                dp[mask] = min(
                    dp[mask],
                    dp[rest] + extra
                );
            }
        }
```

```
        total += dp.back();

        // Collect all target positions in this subtree.
        vector<int> ranks;

        for (const auto& group : groups) {
            ranks.insert(
                ranks.end(),
                group.begin(),
                group.end()
            );
        }

        sort(ranks.begin(), ranks.end());

        // A complete word is always visited before its children.
        if (trie[node].wordIndex != -1) {
            int rank = pos[trie[node].wordIndex];

            auto it = lower_bound(
                ranks.begin(), ranks.end(), rank
            );

            total += it - ranks.begin();
            ranks.insert(it, rank);
        }

        return ranks;
    }

public:
    int minInversions(vector<string>& words, vector<int>& target) {
        int n = words.size();

        trie.clear();
        trie.emplace_back();

        pos.resize(n);
        total = 0;

        // Convert word indices into target positions.
        for (int i = 0; i < n; i++) {
            pos[target[i]] = i;
        }

        // Build the trie.
        for (int i = 0; i < n; i++) {
            int node = 0;

            for (char c : words[i]) {
                int letter = c - 'a';

                if (trie[node].children[letter] == -1) {
                    trie[node].children[letter] = trie.size();
                    trie.emplace_back();
                }

                node = trie[node].children[letter];
            }

            trie[node].wordIndex = i;
        }

        solve(0);
        return total;
    }
};
```

- **Approach:**
  - Construct a prefix tree “trie” from the given words.
  - Convert the target array into positions so that inversions can be identified by comparing the positions of two words.
  - Process the tree recursively, starting from its child subtrees.
  - For each node, calculate inversions caused by placing one child subtree before another.
  - Minimize inversions by finding such order using bitmask dynamic programming.
  - Add inversions caused by complete words that are prefixes of other words, since their traversal order cannot be changed.
  - Return the minimum total inversion count.
- **Code quality:** Code separated into functions and given Solution class is leveraged.
- **Complexity:**
  - Building the “trie” is O(nL) time
    - n = number of words
    - L = maximum word length
  - Bitmask DP has $O(2^d)$ states
    - d = number of children of node
  - Time complexity of the provided implementation is
    - $O(nL(\log n + A) + nA^2 2^A)$
    - where A is the alphabet size.